React Router & Single Page Applications (SPA)

1. What is a Single Page Application (SPA)?

A **Single Page Application** is a web application where:

- The browser loads **one HTML page only once**.
- Navigation between pages does **not reload the browser**.
- Only the required component/content changes.

To achieve this behavior in React, we use a **Router**.

2. Role of React Router

React Router allows us to:

- Define different **paths (URLs)**.
- Map each path to a **component**.
- Navigate between pages **without hard refresh**.

This is the foundation of SPA behavior in React.

3. Navigation Techniques in React

3.1 Link (User-driven navigation)

- Used when navigation happens via user click.
- Prevents default browser reload.
- Replaces anchor (`<a>`) tag behavior.

```
<Link to="/login">Login here</Link>
```

Effect:

- URL updates
- Page does not reload
- State remains preserved

3.2 useNavigate (Logic-driven navigation)

- Used when navigation depends on logic.
- Common use cases:
    - Login success
    - Signup completion
    - Form submission

```
const navigate = useNavigate();
navigate("/login");
```

Decision rule:

- Manual click → `Link`
- Automatic redirect → `useNavigate`

4. NavLink and Conditional Styling

- `NavLink` works like `Link`.
- Provides access to active route state.
- Mainly used to apply **active CSS styles**.

Useful for:

- Navigation bars
- Side menus
- Active tab highlighting

---

5. Traditional Websites vs SPA

5.1 Traditional Website Behavior

In normal websites:

- Each page is a separate HTML file.
- Clicking a link causes:
    - Old page destruction
    - New HTML download
    - CSS and JS files reloaded

You can observe this behavior in:

- Browser DevTools
- Network tab (full reload requests)

5.2 Problems with Hard Refresh

- JavaScript state is destroyed.
- User session data in memory is lost.
- Application must re-fetch data from database.

Example:

- User logs in.
- Navigates to dashboard.
- Page refresh happens.
- Login state disappears.

---

6. Database vs Browser Memory

Important clarification:

- Browser refresh does **not** clear database data.
- But database calls are:
    - Slower
    - Network dependent
    - Costly in performance

This leads to a performance issue in traditional navigation.

---

7. Pre-React Solutions (State Persistence)

Before React, applications used browser-based storage:

- Local Storage (persistent)
- Session Storage (tab-based)
- Cookies (small data storage)

Purpose:

- Store login status
- Preserve user state
- Reduce database dependency

---

8. Browser Architecture Concepts

8.1 Browser Object Model (BOM)

Includes:

- URL
- Tabs
- Navigation controls
- Browser history

8.2 Document Object Model (DOM)

Includes:

- HTML structure
- Elements
- Page content

In traditional websites:

- URL change destroys both BOM-linked content and DOM.
- Entire application resets.

---

9. Why React Solves This Problem

9.1 React SPA Architecture

In React:

- Entire app is bundled together:
    - One HTML
    - One CSS bundle
    - One JS bundle
- Browser downloads everything once.

After initial load:

- Only component rendering changes.
- No file destruction.
- No full reload.

9.2 Browser Caching in React

- Browser caches JS and CSS bundles.
- Repeated requests use local cache.
- Improves speed and efficiency.

This makes React applications faster than traditional multi-page sites.

---

10. Installing React Router

```
npm install react-router-dom
```

This library provides:

- BrowserRouter
- Routes
- Route
- Link
- NavLink
- useNavigate

---

11. BrowserRouter Placement (Core Concept)

- BrowserRouter must be defined **once**.
- Best location: `main.jsx`.
- It provides routing context to the entire app.

main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from "react-router-dom";
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
)
```

Conceptually:

- BrowserRouter acts as a global provider.
- All components gain access to routing features.

---

12. Defining Routes in App.jsx

Routes decide **which component renders for which URL**.

```
import { useState } from 'react'
import { Route, Routes } from "react-router-dom";
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <Routes>
        <Route path="/" element={<SignUp />} />
        <Route path="/login" element={<Login />} />
      </Routes>
    </>
  )
}

export default App
```

---

13. URL Behavior in React

Examples:

- `/`
- `/signup`
- `/login`

Key idea:

- URL changes
- HTML file remains same
- Only component rendering changes

```
<Route path="/signup" element={<SignUp />} />
```

---

14. Common Routing Mistake

Missing `<Routes>`

Symptoms:

- URL updates correctly
- Page content does not change

Reason:

- `Link` updates the URL.
- `Routes` listens for URL changes.
- Without `Routes`, React has nothing to render.

---

15. Route Matching Logic

React Router works sequentially:

1. Reads current URL
2. Matches routes from top to bottom
3. First matching route is rendered
4. No match means no component displayed

```
<Routes>
  <Route path="/" element={<SignUp />} />
  <Route path="/login" element={<Login />} />
</Routes>
```

---

16. Programmatic Navigation Example

Signup Flow Example

```
import { useNavigate } from "react-router-dom";

const navigate = useNavigate();

try {
  // signup successful
  navigate("/login");
} catch (error) {
  // error handling
}
```

Best practice:

- Navigate inside success logic
- Avoid navigation in catch or finally blocks

---

17. Summary & Key Concepts

- React Router enables SPA behavior.
- Hard refresh destroys state; SPA avoids it.
- Link prevents page reload.
- useNavigate handles automatic redirection.
- BrowserRouter belongs in main.jsx.
- Routes are mandatory for rendering.
- React improves performance through caching.
- State preservation is a major advantage of SPAs.
